

TECHNICAL PROJECT DOCUMENTATION

IoT-based AI-Driven Drone Waste Detection & Geospatial Mapping System

*Real-Time Aerial Waste Classification & GPS-Tagged Geospatial Mapping using
ESP32-S3, Deep Learning, and IoT*

Prepared by
Talha Shahzad
Muhammad Yousuf
Shoaib Akhter
Mubashir Ishaq

Ghazi University, Dera Ghazi Khan
Bachelor of Science — Information Technology
6th Semester IoT Project

Document Version 1.0 | June 2026

Table of Contents

1. Executive Summary	3
2. Introduction	3
3. Objectives	4
4. System Architecture	4
5. Hardware Components	5
6. Software & Technologies	5
7. AI Model	6
8. ESP32 Firmware & API Endpoints	6
9. System Workflow — How It Works	6
10. Project Structure	7
11. Installation & Setup Guide	7
12. Demo Modes & Operating Options	8
13. Limitations & Future Enhancements	8
14. Team & Academic Context	9
15. License & Conclusion	9

1. Executive Summary

The IoT-based AI-Driven Drone Waste Detection & Geospatial Mapping System is an embedded computer-vision solution that combines a drone-mounted ESP32-S3 camera module, a deep learning classification pipeline, and GPS-based geotagging to detect and map waste in outdoor environments. The system streams live aerial video from the drone to a local Python server, where each frame is analyzed by a ResNet50-based image classifier to determine whether the captured scene is clean or contains garbage. Whenever a detection occurs, the corresponding GPS coordinates are logged and plotted on an interactive map, giving municipal teams, researchers, or campus authorities a real-time, geospatial view of waste hotspots.

This project was developed as a 6th-semester IoT project at Ghazi University, Dera Ghazi Khan, by a four-member team, and demonstrates the practical integration of embedded systems, wireless communication, deep learning, and geospatial visualization into a single end-to-end pipeline.

It is important to note that this implementation is a Minimum Viable Product (MVP) — it proves the core concept end-to-end rather than representing a finished, production-ready system. Its modular design (separate video source, AI inference, GPS logging, and dashboard layers) means each part can be upgraded or replaced independently as the project matures, as outlined in Section 13.

2. Introduction

2.1 Background

Improper waste disposal is a persistent environmental and public health challenge, particularly in urban and semi-urban areas where manual inspection of large or hard-to-reach areas is time-consuming and inconsistent. Aerial monitoring using drones offers a scalable way to survey large areas quickly, while advances in deep learning make it possible to automatically distinguish clean areas from waste-affected ones directly from video frames.

This project explores the convergence of three technology domains — embedded IoT hardware, convolutional neural networks, and geospatial mapping — to build a low-cost, reproducible prototype for automated waste surveillance.

2.2 Problem Statement

Manual waste-spot identification across large outdoor areas is labour-intensive and does not scale well. There is a need for an automated, low-cost system that can fly over an area, visually identify waste, and record exactly where it was found so that cleanup crews can be directed efficiently.

3. Objectives

- Detect garbage in real time using a drone-mounted camera feed.
- Classify each captured scene as clean or garbage using a trained AI model.
- Record GPS coordinates at the exact moment a detection occurs.
- Visualize waste locations on an interactive map using colour-coded markers (green for clean, red for garbage).
- Provide a live monitoring dashboard accessible through a standard web browser.

4. System Architecture

The system follows a simple client–server architecture in which the ESP32-S3 camera module acts as a wireless Access Point and video/GPS data source, while a Python Flask server running on a laptop performs inference, GPS logging, and dashboard rendering.

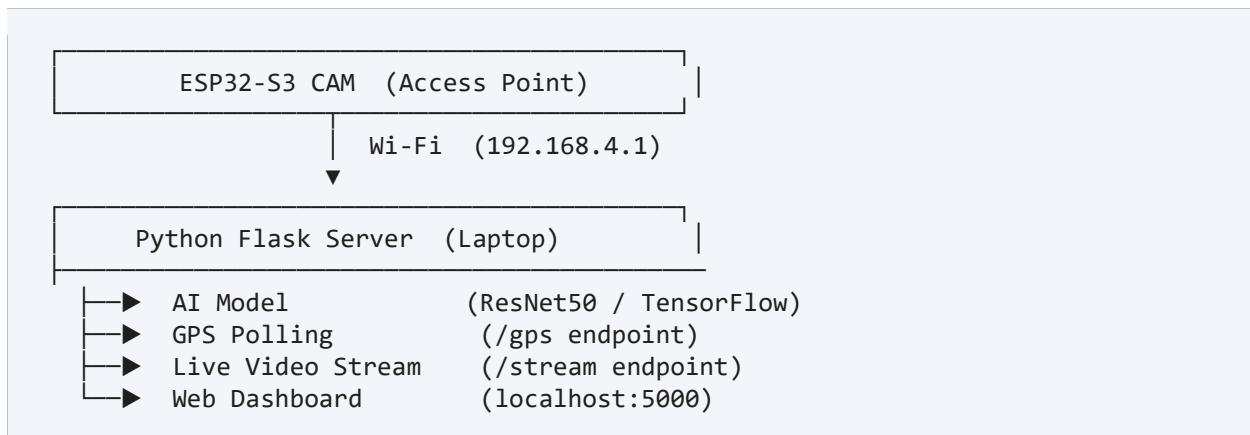


Figure 1 — High-level system architecture

4.1 Data Flow Description

1. The ESP32-S3 CAM boots and starts a dedicated Wi-Fi Access Point (SSID: ESP32_CAM).
2. The host laptop connects to this access point, joining the same local network as the drone.
3. The Flask server continuously reads the MJPEG stream exposed at the /stream endpoint.
4. Every 10th frame is passed to the ResNet50 classifier to determine clean vs. garbage.
5. In parallel, the server polls the /gps endpoint every 5 seconds to retrieve the current location.
6. When a garbage detection coincides with a GPS reading, the coordinate is stored and rendered on the Leaflet.js map as a red marker; clean readings are rendered as green markers.
7. All results are surfaced live on the web dashboard at localhost:5000.

5. Hardware Components

The hardware layer is intentionally minimal, relying on a single camera-equipped microcontroller and an external GPS module to keep the prototype lightweight enough for drone mounting.

Component	Description
GOOUUU ESP32-S3-CAM	Main microcontroller board with an integrated OV3660 camera sensor; handles Wi-Fi access point hosting, video capture, and streaming.
NEO-6M GPS Module	U-blox GPS receiver with an external antenna, used to determine real-time drone location via NMEA sentences.
Jumper Wires	Used for GPIO-level connections between the GPS module and the ESP32-S3 board.

6. Software & Technologies

Technology	Purpose
Python 3.11	Core language for the backend inference and web server.
TensorFlow / Keras	Training and running the deep-learning waste-classification model.
ResNet50	Pre-trained convolutional neural network used via transfer learning for image classification.
Flask	Lightweight web server exposing the REST API and dashboard.
OpenCV	Frame decoding and image pre-processing from the MJPEG stream.
Arduino IDE	Firmware development environment for the ESP32-S3.
TinyGPS++	Parsing NMEA sentences from the NEO-6M GPS module on the ESP32.
Leaflet.js	Rendering the interactive map UI with clean/garbage markers.

7. AI Model

7.1 Model Architecture

The classification engine is built on ResNet50, a deep convolutional neural network, applied through transfer learning rather than training from scratch. This approach reuses image features already learned on a large, general-purpose dataset and fine-tunes the network for the binary clean/garbage classification task, substantially reducing the amount of training data and compute required.

7.2 Dataset

The model was trained on the publicly available Clean/Dirty Road Classification dataset hosted on Kaggle, which contains labelled images for two classes: clean and garbage.

7.3 Training & Performance

- Training environment: Google Colab (GPU-accelerated).
- Classes: clean / garbage.
- Validation accuracy: approximately 80%.

This level of accuracy is appropriate for a semester-length academic prototype and demonstrates that transfer learning is a viable, low-resource approach to aerial waste classification; it also points to clear opportunities for improvement, discussed in Section 13.

8. ESP32 Firmware & API Endpoints

The ESP32-S3 firmware exposes two HTTP endpoints that the Flask server consumes over the local Wi-Fi access point.

Endpoint	Method	Description
/stream	GET	Returns a live MJPEG video stream from the onboard OV3660 camera.
/gps	GET	Returns the current GPS coordinates as a JSON payload, parsed from the NEO-6M module via TinyGPS++.

9. System Workflow — How It Works

8. The ESP32-S3 starts up as a Wi-Fi Access Point named ESP32_CAM.
9. The operator's laptop connects to this access point.
10. The Python Flask server reads the live MJPEG stream from the ESP32.
11. Every 10 frames, the AI model classifies the current scene as clean or garbage.
12. GPS coordinates are polled from the ESP32 every 5 seconds.
13. Classification results and location data are displayed on the live web dashboard at localhost:5000.

14. Detected waste locations are plotted on an interactive Leaflet.js map for review.

10. Project Structure

```
IoT-Drone-Waste-Detection-System/  
├── ..._ESP32_Code/  
│   ├── IoT_Drone_Project.ino    - Main ESP32 firmware  
│   ├── TinyGPSPPlus.h          - GPS library header  
│   └── TinyGPSPPlus.cpp         - GPS library source  
├── ..._Python_Model/  
│   └── server.py                - Flask server + AI inference  
├── .gitignore  
└── README.md
```

Figure 2 — Repository folder structure

11. Installation & Setup Guide

11.1 ESP32 Firmware Setup

15. Open IoT_Drone_Project.ino in the Arduino IDE.
16. Select the board: ESP32S3 Dev Module.
17. Upload the firmware via the TTL port.
18. Switch to the OTG port to open the Serial Monitor for debugging.

11.2 Python Server Setup

Run the following commands in sequence to prepare and launch the backend server:

```
# Create virtual environment  
py -3.11 -m venv venv311  
venv311\Scripts\activate  
  
# Install dependencies  
pip install flask tensorflow opencv-python numpy requests  
  
# Run server  
python server.py
```

Figure 3 — Python environment setup and server launch commands

11.3 Connecting & Running

19. Connect the laptop's Wi-Fi to the ESP32_CAM network (password: 12345678).
20. Open a browser and navigate to <http://localhost:5000>.

21. The live dashboard will display the video feed, classification results, and the geospatial map.

12. Demo Modes & Operating Options

To make the system practical to demonstrate and test in different settings — including indoor presentations where flying a drone is not feasible — the pipeline supports two interchangeable video-source modes. Both modes feed the same MJPEG-style stream into the unchanged Flask server, AI inference, and dashboard layers, so only the camera source changes between them.

12.1 Mode A — ESP32-S3 Built-in Camera (Primary / Drone-Mounted)

This is the original, drone-mounted configuration described throughout this document. The ESP32-S3 CAM module hosts its own Wi-Fi access point and exposes the `/stream` and `/gps` endpoints directly from the onboard OV3660 camera and the connected NEO-6M GPS module. This mode is intended for real outdoor flights and field trials.

12.2 Mode B — Mobile Phone Camera (IP Webcam Alternative)

For quick demos, indoor testing, or situations where the ESP32 hardware or a drone is not available, a smartphone can be used in place of the ESP32-S3 camera. An IP camera app (for example, IP Webcam on Android) turns the phone into a Wi-Fi video server, streaming MJPEG video over the local network in the same format the Flask server already expects. The server's video-source URL is simply pointed at the phone's IP camera address instead of the ESP32's `/stream` endpoint, and the AI classification and dashboard continue to work unchanged. GPS coordinates in this mode can be taken from the phone's own location services or entered manually for demo purposes.

This dual-mode setup means the system can be shown end-to-end — live classification, mapping, and dashboard — without needing an actual drone flight, making it well suited for project evaluations, classroom demonstrations, and quick functional testing.

13. Limitations & Future Enhancements

13.1 Current Limitations

- Binary classification only (clean vs. garbage) — does not distinguish between waste types or severity.
- Validation accuracy of approximately 80% leaves room for misclassification, particularly in cluttered or low-light scenes.
- Sampling every 10th frame may miss very brief or fast-moving detections.
- The ESP32 access point has a limited Wi-Fi range, constraining how far the drone can fly from the operator's laptop.

- GPS polling at 5-second intervals introduces a small lag between visual detection and location logging.

13.2 Proposed Future Enhancements

- Expand the model to multi-class waste categorization (e.g., plastic, organic, construction debris).
- Integrate autonomous flight planning so the drone can survey a defined area without manual piloting.
- Move from a local Flask dashboard to a cloud-hosted service for remote, multi-user access.
- Add a companion mobile app for field crews to receive real-time alerts and directions to detected waste sites.
- Increase classification accuracy through a larger, more diverse training dataset and model fine-tuning.

14. Team & Academic Context

14.1 Team Members

Name	Notes
Talha Shahzad	Team member
Muhammad Yousuf	Team member
Shoaib Akhter	Team member
Mubashir Ishaq	Team member

14.2 Academic Context

This project was developed as a 6th-semester IoT project at the undergraduate level, within the Bachelor of Science in Information Technology programme at Ghazi University, Dera Ghazi Khan.

15. License & Conclusion

15.1 License

This project is published for academic purposes only.

15.2 Conclusion

The IoT-based AI-Driven Drone Waste Detection & Geospatial Mapping System successfully demonstrates an end-to-end pipeline that links embedded hardware, wireless communication,

deep learning, and geospatial visualization into a single working prototype. By combining an ESP32-S3 camera module (or, alternatively, a mobile-phone IP camera for demo purposes), a ResNet50-based classifier, and GPS-based geotagging, the system offers a practical, low-cost foundation for automated aerial waste monitoring.

As stated throughout this document, the current build is a Minimum Viable Product (MVP): it validates the core idea rather than delivering a finished product. Its modular structure makes it straightforward to extend — whether by adding multi-class waste detection, autonomous flight, cloud hosting, or mobile alerts — toward a more advanced, production-grade environmental monitoring solution in future iterations.

— *End of Document* —